

Floating Point Representation

Chapter 6

Stats 102A: Introduction to Computational Statistics with R

UCLA



1 Approximate Storage of Numbers

2 Floating Point Representation

Section 1

Approximate Storage of Numbers

Real Numbers

The set of **real numbers** $\mathbb{R}$ has many definitions and constructions.

Intuitively, $\mathbb{R}$ can be thought of as all the points on an infinitely long line (the **number line** or **real line**), where integer points are equally spaced.

There are infinitely many real numbers. In fact, there are infinitely many real numbers between any two distinct real numbers.

Side Note: If you want to learn a mathematically rigorous construction of $\mathbb{R}$, you can take Math 131 (Real Analysis).

Properties of Real Numbers

The real numbers have many properties that we take for granted when working with numbers. Some key ones are given below.

Let a , b , and c denote real numbers.

Associativity:

$$\begin{aligned}(a + b) + c &= a + (b + c) \\ (a \times b) \times c &= a \times (b \times c)\end{aligned}$$

Distributive Property:

$$a \times (b + c) = (a \times b) + (a \times c)$$

Commutative Property:

$$\begin{aligned}a + b &= b + a \\ a \times b &= b \times a\end{aligned}$$

Side Note: These properties are consequences of the fact that $\mathbb{R}$ forms a

field. If

you want to learn more about rings and fields, you can take Math 5/35

Decimal Representation

Another property of $\mathbb{R}$ is that every real number has a **decimal representation**, i.e., any real number r can be expressed as

$$r = (-1)^s \sum_k a_k 10^k,$$

where $s \in \{0, 1\}$ determines the sign of r , $a_k \in \{0, 1, 2, \dots, 9\}$ are the digits of r , and 10 is the base (or **radix**) of the number system.

A shorthand for the full summation representation is to represent r by its a_k coefficients, with a dot (decimal point) to denote the boundary between non-negative and negative powers of 10.

For example, the decimal number 5413.29 can be expressed as

$$5413.29 = 5 \times 10^3 + 4 \times 10^2 + 1 \times 10^1 + 3 \times 10^0 + 2 \times 10^{-1} + 9 \times 10^{-2}.$$

Scientific Notation

An alternative expression to decimal representation is **scientific notation**, i.e., any real number r can be expressed as

$$r = (-1)^s m \times 10^n,$$

where $s \in \{0, 1\}$ determines the sign of r and m and n are integers. The integer n is called the **exponent** (or **order of magnitude**), and the integer m is called the **mantissa** (or **significand**).

Typically, scientific notation is **normalized** such that the mantissa is $1 \leq m < 10$. That is, r is in **normalized scientific notation** if r is expressed as

$$r = (-1)^s a_0.a_1a_2a_3 \cdots \times 10^n,$$

where $a_0, a_1, a_2, a_3, \dots \in \{0, 1, 2, \dots, 9\}$ are digits with $a_0 \neq 0$.

Scientific Notation

For example, non-normalized scientific notation for 5413.29 could be

$$5413.29 = 541329 \times 10^{-2},$$

while normalized scientific notation is

$$5413.29 = 5.41329 \times 10^3.$$

Scientific notation is often useful for representing very small or very large numbers in a compact way, such as $6.02214076 \times 10^{23}$ or $9.10938356 \times 10^{-31}$.

Infinite Decimal Representation

Humans often represent real numbers and perform arithmetic calculations using decimal representation. However, most (i.e., nearly *all*) real numbers have an infinite decimal representation.¹

For example, all irrational numbers, such as $\sqrt{2}$, π , or e , have infinite decimal representations. Even many rational numbers, such as $1/3$, $1/7$, or $1/9$, have infinite decimal representations.

Thus, every time we write a decimal representation of a real number with infinite decimal representation on paper, it is an *approximation* to the true value.

However, using approximations has profound implications for the properties of real numbers that we take for granted.

¹**Side Note:** Technically, *every* real number has an infinite decimal representation, since we can add infinite 0's or 9's to any terminating decimal. However, by convention, any infinite decimal representation with repeating 0's or 9's at the end are considered **finite** decimal representations and we represent only the non-repeating digits.

Approximate Storage of Numbers

Computers are unable to represent real numbers with infinite precision. Computers do not have infinite storage, so they use a set number of bits to represent a number. This results in many numbers being represented by an approximate representation.

Computers technically store numbers in a binary representation (base 2 instead of base 10), but the effects of approximation can be seen even using decimal representation.

Breaking the Rules

Many simple calculators can only show 8 digits on the screen.

Suppose we compute $10/3$ on a simple calculator.

- The result will show 3.3333333, which is the best representation of $10/3$ that the calculator can show with 8 digits.

Next we take 3.3333333 and multiply by 3.

- The result will show 9.9999999, which is not equal to 10.

On the other hand, what if we computed $10*3$ then divided by 3?

- The result will show 10, which is correct. ## Breaking the Rules

Even in R, which is much more powerful than a simple calculator, the approximate storage of numbers can cause unexpected results.

```
x <- ((4 / 5) * 3) * (5/4)
x
```

Breaking the Rules

Many simple calculators can only show 8 digits on the screen.

Suppose we compute $10/3$ on a simple calculator.

- The result will show 3.3333333, which is the best representation of $10/3$ that the calculator can show with 8 digits.

Next we take 3.3333333 and multiply by 3.

- The result will show 9.9999999, which is not equal to 10.

On the other hand, what if we computed $10*3$ then divided by 3?

- The result will show 10, which is correct. ## Breaking the Rules

Even in R, which is much more powerful than a simple calculator, the approximate storage of numbers can cause unexpected results.

```
x <- ((4 / 5) * 3) * (5/4)
x
```

Breaking the Rules

More examples:

```
0.1 == 0.3 / 3
```

```
## [1] FALSE
```

```
0.1 + 0.2 == 0.3
```

```
## [1] FALSE
```

```
seq(0, 1, by = 0.1)[4] == 0.3
```

```
## [1] FALSE
```

Breaking the Rules

```
options(digits = 20) # Or use print(x,digits=20)
```

```
0.1
```

```
## [1] 0.100000000000000000555
```

```
0.2
```

```
## [1] 0.200000000000000000111
```

```
0.1 + 0.2
```

```
## [1] 0.3000000000000000004441
```

```
0.3
```

```
## [1] 0.299999999999999999889
```

Breaking the Rules

Another interesting example:²

```
x <-  
  c(0.3, 0.4 - 0.1, 0.5 - 0.2, 0.6 - 0.3, 0.7 - 0.4)  
x  
  
## [1] 0.3 0.3 0.3 0.3 0.3
```

Question: How many of these do you think are distinct values? How many distinct values *should* there be?

²**Side Note:** This example comes from “The R Inferno” by Patrick Burns:
<https://www.burns-stat.com/documents/books/the-r-inferno/>

Breaking the Rules

They should all be equal, but there are *three* distinct values!

```
x <-  
  c(0.3, 0.4 - 0.1, 0.5 - 0.2, 0.6 - 0.3, 0.7 - 0.4)  
x
```

```
## [1] 0.3 0.3 0.3 0.3 0.3
```

```
unique(x) # Remove duplicate values
```

```
## [1] 0.3 0.3 0.3
```

```
print(x, digits = 20)
```

```
## [1] 0.29999999999999998890 0.300000000000000004441
```

```
## [3] 0.29999999999999998890 0.29999999999999998890
```

```
## [5] 0.29999999999999993339
```


Binary Representation

Rather than a decimal (base 10) representation, computers represent numbers using a **binary** (or **base 2**) representation, where each bit (or digit) is either a 0 or 1.

In binary representation, any real number r can be expressed as

$$r = (-1)^s \sum_k b_k 2^k,$$

where $s \in \{0, 1\}$ determines the sign of r , $b_k \in \{0, 1\}$ are the digits of r (in binary), and 2 is the base (or radix).

Note: When ambiguous, we will write $(r)_{10}$ to represent the decimal representation and $(r)_2$ to represent the binary representation.

Binary Representation

Any integers can be represented exactly with binary representation ($b_0 = 0$ for even integers and $b_0 = 1$ for odd integers).

For example, the first 16 non-negative integers (using 4 bits/digits) are represented below:

Decimal	Binary	Decimal	Binary
0	0000	8	1000
1	0001	9	1001
2	0010	10	1010
3	0011	11	1011
4	0100	12	1100
5	0101	13	1101
6	0110	14	1110
7	0111	15	1111

Finite Binary Representation

A real number has a finite binary representation if and only if it can be written as a fraction such that the denominator is a power of 2 (i.e., it has the form 2^n for some non-negative integer n). For example:

Decimal	Full Binary Representation	Binary
0.5	1×2^{-1}	0.1
0.25	$0 \times 2^{-1} + 1 \times 2^{-2}$	0.01
0.125	$0 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3}$	0.001
0.375	$0 \times 2^{-1} + 1 \times 2^{-2} + 1 \times 2^{-3}$	0.011

Infinite Binary Representation

Just like in decimal representation, most real numbers have an infinite binary representation.

For example, the decimal number 0.1 has the binary representation given by

$$(0.1)_{10} = (0.00011001100110011\dots)_2 = (0.0\overline{0011})_2,$$

where the bar represents a repeating pattern.

Since computers can only use finitely many bits to store numbers, the binary representation is truncated at a certain number of digits, so the decimal number 0.1 in a computer will always be an approximation to the true value.

This is why `0.1 + 0.2 == 0.3` returns `FALSE` in R: The binary approximation of 0.1 plus the binary approximation of 0.2 is not equal to the binary approximation of 0.3.

Section 2

Floating Point Representation

IEEE 754 Standard

Most programming languages, including R, use **floating point representation**, which is a binary variation on scientific notation.

Modern computers use the **IEEE 754** standard for floating point representation. This is implemented at the hardware level on the actual computer chip itself.

The IEEE 754 standard was revised in 2008, but (for educational purposes) we will discuss the original 1985 specification, which is still fully contained in the 2008 revision.

More on IEEE 754: https://en.wikipedia.org/wiki/IEEE_754

Side Note: IEEE stands for Institute of Electrical and Electronics Engineers.

Floating Point Representation

Floating point representation expresses numbers in a normalized binary notation. The real number r is expressed as

$$r = (-1)^s 1.b_1 b_2 \dots b_m \times 2^n,$$

where $s \in \{0, 1\}$ determines the **sign** of r , $b_1, b_2, \dots, b_m \in \{0, 1\}$, and n is an integer.

As in scientific notation in base 10, the fraction part $b_1 b_2 \dots b_m$ is the **mantissa** (or **significand**) and n is the **exponent**.

To store this representation, the computer needs to allocate bits for the sign s , the m digits of the significand, and the exponent n (also represented in binary).

Note: The leading 1 before the “decimal”³ point is not stored in the computer, as it is implied for all numbers.

³**Side Note:** The dot is technically called a **bicimal point** or **radix point**, since the number is not in decimal representation.

Example of Floating Point Representation

Consider the decimal number 6.375.

In binary, $(6.375)_{10} = (110.011)_2$.

In floating point representation, we normalize the binary representation to

$$110.011 = 1.10011 \times 2^2.$$

Just like scientific notation, moving the “decimal” point to the left increases the exponent, and moving the point to the right decreases the exponent.

Floating Point Precision

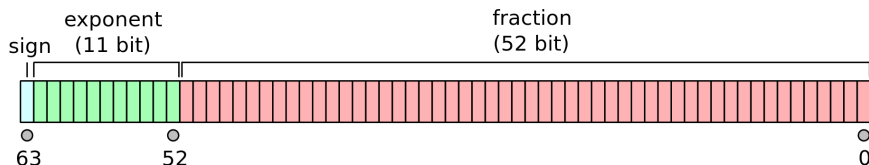
Computers use different levels of **precision** to store numbers, depending on how much storage it will use, i.e., how many bits it will allocate to store a number. Higher levels of precision will produce a better approximation, but they will require more memory.

With each level of precision, the computer allocates a certain number of bits to store the sign, the mantissa, and the exponent of the floating point number.

Precision	Sign	Exponent	Mantissa	Total
Single	1	8	23	32
Double	1	11	52	64
Long Double	1	15	64	80

Double Precision

R (and most other languages) uses 64-bit **double** precision, which is why non-integer numeric values in R are of type **double**.



Source: https://en.wikipedia.org/wiki/File:IEEE_754_Double_Floating_Point_Format.svg

Exponents and Exponent Bias

The 11 bits allocated for the exponent allows us to represent $2^{11} = 2048$ different integer exponent values.

Two integers are reserved for special cases, which leaves 2046 integers to represent both positive and negative (and 0) exponents.

Thus, the possible exponents range from -1022 to 1023 . The integers in this range are stored as 11-bit binary representations of the numbers 1 to 2046: 00000000001 to 11111111110.

This creates the **exponent bias** of 1023: The stored value of the exponent is off from the actual value by $1023 = 2^{10} - 1$.

To represent the actual exponent n , we add the exponent bias 1023 (or $1024 - 1$) to it and store the binary representation $(n + 1023)_2$.

Exponents and Exponent Bias

The table below shows the actual exponent represented and the corresponding stored integer, in decimal and 11-bit binary forms.

Exponent	Decimal	Binary
Special	0	00000000000
-1022	1	00000000001
-1021	2	00000000010
$\vdots$	$\vdots$	$\vdots$
1023	2046	11111111110
Special	2047	11111111111

Special Exponents

Special cases:

- The exponent of 2047, or $(1111111111)_2$, represents ∞ (Inf) if the mantissa is all zeroes and Not A Number (NaN) otherwise.
- The exponent of 0, or $(0000000000)_2$, is used to represent **denormalized (subnormal)** numbers, which are numbers less than 1.0×2^{-1022} (i.e., the non-stored leading number in the floating point representation is 0 instead of the implicit 1).⁴

⁴**Side Note:** Subnormal numbers are used when the result of a calculation is smaller than the smallest number that can be represented by the computer. More information here: https://en.wikipedia.org/wiki/Arithmetic_underflow

Example of Double Precision

Consider the decimal number 6.375, which is 110.011 in binary.

The floating point representation is 1.10011×2^2 .

Adding exponent bias, the stored exponent is $2 + 1023 = 1025$.

In 11-bit binary, 1025 is 10000000001.

So, in double precision, 6.375 is stored as:

- The sign: 0
- The exponent: 10000000001
- The mantissa:

10011000

More compactly: 0|10000000001|10011000...000

Machine Epsilon

Because of the finite precision, floating point numbers have a **granularity**: There are gaps between consecutive floating point numbers.

We can consider the distance ε between 1 and the next smallest representable number as the smallest positive floating point number such that $1 + \varepsilon \neq 1$. This number ε is called **machine epsilon**.

When representing numbers with floating point representation, machine epsilon represents an upper bound on the relative error between the exact number and its floating point approximation.

Machine Epsilon

In double precision, the next smallest representable number greater than 1 is

$$1 + \varepsilon = 1.000 \cdots 001 = 1 + 2^{-52},$$

so $\varepsilon = 2^{-52}$.

Machine epsilon is a stored quantity in R:

```
.Machine$double.eps
```

```
## [1] 2.220446e-16
```


Rounding

For numbers with an exact representation that requires more than 52 bits in the mantissa, we would need to approximate the number by rounding to the nearest 52-bit mantissa representation.

Suppose a number r is represented as

$$r = (-1)^s 1.b_1 b_2 \dots b_{52} b_{53} \dots \times b^n.$$

The digit b_{53} represents the first digit beyond the 52-bit mantissa.

- If $b_{53} = 0$, round down by truncating b_{53} and all digits beyond.
- If $b_{53} = 1$ and at least one digit beyond is nonzero, round up by adding 1 to b_{52} and adjust the mantissa as necessary.
- If $b_{53} = 1$ and all digits beyond are zero, then round up if $b_{52} = 1$ and round down if $b_{52} = 0$.

This rounding rule is analogous to rounding decimal numbers based on if a digit is 5 and if the preceding digit is even or not.

More information: https://en.wikipedia.org/wiki/Rounding#Round_half_to_even

Testing Equality

Since testing for equality with `==` does not allow for rounding error from floating point representations, the `all.equal()` function is almost always preferred for testing (near) equality of numbers.

The optional `tolerance` argument allows for negligible differences. The default value is `tolerance = sqrt(.Machine$double.eps)`.

```
0.1 + 0.2 == 0.3
```

```
## [1] FALSE
```

```
all.equal(0.1 + 0.2, 0.3)
```

```
## [1] TRUE
```

Testing Equality

Note: If you want to use the `all.equal()` function inside of a conditional statement that requires a logical value either `TRUE` or `FALSE`, it is preferable to use `identical()` outside of `all.equal()`. This ensures that the output will be a single logical value.

```
all.equal(0.5 - 0.2, 0.4 - 0.1)
```

```
## [1] TRUE
```

```
all.equal(0.5, 0.1)
```

```
## [1] "Mean relative difference: 0.8"
```

```
identical(all.equal(0.5, 0.1), TRUE)
```

```
## [1] FALSE
```

Takeaways

Some practical things to keep in mind:

- R generally is good at hiding approximation error by printing nicely, but do not forget that the approximation error is still there.
- Whenever floating point operations are done, even basic ones, you should assume that there will be approximation error.
- Use `all.equal()` with `identical()` instead of `==` to compare floating point numbers.
- Do not confuse approximation error with an error. An error is when a computation is incorrectly performed. Approximation error is when there is visible noise resulting from the finite representation of numbers. It is approximation error, not an error, when one-third is represented as 33%.